# ScamTracer AI: A Unified Framework for Real-Time Scam Engagement, Behavioral Analysis, and Graph-Based Intelligence

***Abstract*—Online scams increasingly operate as adaptive conversations rather than isolated malicious messages. Attackers establish trust, manufacture urgency, introduce persuasive financial narratives, request sensitive information, and modify their strategy in response to the target. This evolution creates a gap between conventional detection systems, which often classify individual messages or static artifacts, and investigation-oriented systems, which require longitudinal behavioral evidence, technical indicators, and relationship analysis. This extended manuscript develops the ScamTracer AI framework from the three-page draft into a detailed research design. The work synthesizes literature on digital forensics, messaging-network analysis, conversational scam detection, linguistic deception, anomaly detection, imbalanced fraud classification, honeypots, and graph analytics. The proposed architecture combines a privacy-constrained conversational engagement engine, stage-aware feature extraction, statistical and machine-learning analysis, entity resolution, graph-based intelligence, and forensic evidence generation. Particular attention is given to an R-centered analytical implementation, enabling data cleaning, exploratory analysis, statistical testing, visualization, classification, anomaly detection, and network analysis within a reproducible research workflow. The manuscript does not claim completed deployment results; instead, it specifies measurable experiments, evaluation criteria, privacy controls, threat assumptions, and limitations required to validate the system. The resulting design treats a scam as an evolving interaction whose messages, entities, timing, infrastructure, and relationships can jointly contribute to an evidence-oriented intelligence record.**

***Index Terms*—Scam detection, conversational intelligence, scambaiting, behavioral analysis, digital forensics, graph analytics, anomaly detection, natural language processing, privacy-preserving AI, WhatsApp.**

## I. Introduction

The communication model of online fraud has changed substantially with the growth of instant messaging, social platforms, online payment systems, and disposable digital infrastructure. A scam can begin with an apparently ordinary greeting and gradually develop into a sequence of trust-building, urgency creation, financial persuasion, and compliance requests. The investigative problem is therefore not limited to deciding whether a single message is malicious. A useful system must preserve the temporal order of interactions, distinguish participants, identify technical and financial entities, quantify behavioral changes, and connect apparently separate conversations when they share infrastructure or behavioral signatures.

The literature underlying ScamTracer provides evidence for each part of this requirement. WhatsApp network-forensics research has demonstrated that packet-level analysis can be used to isolate communication attributes relevant to investigation, while broader computer-forensics surveys emphasize acquisition, preservation, analysis, and reporting of heterogeneous digital artifacts [1], [5], [15]. Technical-support-scam research demonstrates that scam campaigns can be measured through web infrastructure, social-media activity, backlinks, and machine-learning features rather than only message text [2], [63], [68]. Conversational work further shows that scam interactions evolve through identifiable linguistic stages, making a time-aware representation more appropriate than a static bag of scam keywords [7], [30]–[32].

At the behavioral level, fraud research has repeatedly encountered severe class imbalance. Hashemi et al. investigate weight tuning and gradient-boosted ensembles for banking fraud, while Irofti et al. survey one-class and graph-oriented anomaly detection methods for financial networks [4], [8], [40], [41]. Wang and Zhu show how fine-grained co-occurrences can enrich behavioral representations by using heterogeneous network embeddings, suggesting that relationships among attributes can carry information that isolated transaction fields cannot [11], [47], [49]–[51]. Graph-computing studies similarly argue that fraud and financial crime can be represented as network phenomena in which the relevant signal may lie in connections among entities rather than in one record [10], [13], [55].

The proposed ScamTracer AI framework extends these observations into a unified pipeline. It does not treat conversational engagement as an end in itself. Engagement is a controlled data-acquisition mechanism bounded by privacy, safety, and evidence-preservation requirements. The system should collect only the indicators required for investigation, avoid exposing genuine personal information, maintain a distinction between observed evidence and model-generated inference, and retain provenance for every analytical conclusion. This distinction is important because a phone number, payment identifier, URL, linguistic pattern, or graph connection may be an indicator of association without being proof of identity or criminal responsibility.

The principal contribution of this extended manuscript is a detailed research architecture connecting five layers: conversational interaction, feature engineering, statistical and machine-learning analysis, graph intelligence, and forensic reporting. The design also specifies how R can serve as the analytical core. Rather than using R only for visualization, the framework uses R for data ingestion, transformation,

feature construction, descriptive statistics, hypothesis testing, classification, anomaly analysis, network construction, and report generation. This creates a direct methodological bridge between the proposed cybersecurity system and an R-based data-science implementation.

## II. Literature-Derived Research Foundation

The original draft organizes the literature into four domains: digital forensics and network indicators; attacker engagement and linguistic dynamics; behavioral analysis and machine learning; and graph-based fraud detection. The expanded analysis retains this organization but makes the dependencies between domains explicit.

### A. Digital Forensics and Communication Evidence

Javed et al. survey computer-forensic domains including operating-system, file-system, live-memory, web, email, network, and multimedia forensics. Their discussion emphasizes that useful evidence can originate from heterogeneous digital objects and that investigation requires appropriate acquisition, examination, analysis, and reporting processes [5], [16]–[18]. The survey also identifies privacy concerns and the increasing difficulty introduced by cloud, distributed, and emerging computing environments. For ScamTracer, these observations motivate an evidence layer rather than a classifier-only architecture.

Kao et al. specifically examine WhatsApp network traffic and propose packet filtering and rule extraction to isolate suspicious IP-related attributes from large traffic collections [1], [15], [61]. The important architectural implication is that network indicators and conversation content should be treated as complementary evidence streams. A system that observes only message semantics can miss infrastructure reuse; a system that observes only network telemetry can miss the persuasive sequence that explains why an interaction is suspicious. ScamTracer therefore represents network observations as timestamped indicators that can later be joined with conversation-level entities.

The forensic literature also emphasizes evidence integrity. A research prototype intended for cybercrime investigation must preserve source timestamps, extraction time, transformation steps, and analyst actions. Each derived feature should be reproducible from an immutable or integrity-protected source representation. The proposed evidence record consequently contains a source identifier, event timestamp, processing timestamp, extraction method, feature name, value, confidence, and provenance metadata. Such a structure supports later auditing and helps separate the original observation from downstream analytical interpretations.

### B. Conversational Engagement and Honeypot Principles

Honeypots are traditionally used to attract adversarial activity into controlled environments so that defenders can observe behavior without exposing production assets. Uchibori et al. demonstrate a blockchain-oriented honeypot in which attackers can be attracted without holding real crypto-assets, and they combine engagement with clustering of observed actions [3], [21]–[24]. This principle transfers naturally to conversational fraud: the system can provide an artificial interaction surface designed to reveal tactics while minimizing the information available about the real user.

The conversational version is more complex because language is adaptive. A scammer may stop responding if the simulated victim becomes implausible, may change the requested payment mechanism, or may escalate pressure when the target hesitates. The engagement engine must therefore maintain conversational state and enforce a bounded persona. The persona should contain no real name, address, phone number, financial account, authentication secret, or other sensitive identifier. Any generated detail should be synthetic and non-actionable.

Hossain et al. propose an AI-in-the-loop framework combining real-time scam detection, LLM-generated engagement, utility-based response selection, federated learning, and privacy controls [6], [25]–[28]. Their work provides a direct conceptual basis for a safety-aware engagement layer. ScamTracer adopts the separation between detection and engagement: a conversation must first cross a risk threshold before automated engagement is activated, and candidate responses must be screened for privacy and harm before transmission.

### C. Linguistic Dynamics of Scam Conversations

Li et al. analyze 276,762 messages from 49 authentic scam cases using the COLD framework and identify five sequential stages: Trust Cultivation, Risk Fabrication, Persuasive Pitching, Compliance Induction, and Repeated Exploitation [7], [29], [30], [34], [35]. Their results support a central design assumption of ScamTracer: the same word or feature may have different investigative meaning depending on the stage of an interaction.

A stage-aware system therefore should not reduce an entire conversation to a single term-frequency vector. Instead, it should create a sequence of windows. For each window, ScamTracer can calculate linguistic, behavioral, and entity-level features. A message containing a payment identifier during a neutral administrative conversation may not carry the same risk as a payment request that appears after an extended trust-building phase. Similarly, repeated use of urgency terms after a victim expresses hesitation can represent escalation that is invisible to a static classifier.

The COLD literature also motivates psycholinguistic features. LIWC-based analysis can capture cognitive processes, emotional language, social processes, affiliation, achievement, power, money, risk, certainty, and tentativeness [7], [32]–[34]. ScamTracer does not interpret these categories as direct psychological diagnoses. They are observable linguistic features that may contribute to an aggregate risk model. This distinction prevents the system from converting linguistic patterns into unsupported claims about an individual's mental state or identity.

### D. Behavioral Anomaly Detection and Imbalanced Learning

Fraud datasets are typically imbalanced because legitimate transactions or messages greatly outnumber confirmed fraudulent examples. Hashemi et al. address this problem through

class-weight tuning, Bayesian optimization, LightGBM, XGBoost, CatBoost, ensemble learning, and deep-learning comparisons [4], [37]–[39]. The key lesson for ScamTracer is methodological: evaluation should not rely on accuracy alone. Precision, recall, F1 score, Matthews correlation coefficient, PR-AUC, ROC-AUC, and calibration should be reported together, especially when false negatives and false positives have different operational costs.

Irofti et al. review one-class classification, network anomaly detection, clustering, attributed graphs, and deep anomaly-detection approaches [8], [40]–[43], [55]. This is relevant when confirmed scam labels are scarce. Instead of requiring every conversation to be labeled, ScamTracer can learn normal behavioral patterns and flag deviations for analyst review. For example, unusually high message frequency, repeated requests across many targets, rapid switching among payment identifiers, or synchronized URL reuse can be modeled as behavioral deviations.

Liu et al. propose an integrated anomaly-detection method for massive system logs using K-prototype clustering, k-nearest-neighbor classification, and a clustering-filtering-refinement pipeline [12], [45], [46]. The multi-stage principle is useful for ScamTracer because raw messaging events can first be clustered into behavioral profiles, filtered for candidate anomalies, and then refined using supervised or rule-based evidence. This reduces the need to apply expensive models to every event.

### *E. Graph-Based Fraud Intelligence*

Wang and Zhu construct fine-grained co-occurrence relationships among transactional attributes using a knowledge graph and heterogeneous network embedding [11], [47]–[50], [53]. Their work suggests that entities and attributes should not be flattened prematurely. A payment identifier can be linked to a phone number, which can be linked to a URL, which can be linked to another conversation. The resulting graph may expose repeated infrastructure that is not obvious when conversations are analyzed independently.

Kurshan et al. discuss graph computing for financial crime and fraud detection and identify graph neural networks, adaptive graph methods, implementation complexity, and operational integration as important considerations [13], [51], [54]. Irofti et al. similarly review graph anomaly detection, community detection, attributed networks, and anomaly scoring [8], [55], [56]. Modepalli's network-analytics framework emphasizes network construction, community detection, centrality measures, anomaly detection, and integration with risk management [10].

Chornyi and Stopochkina examine information-flow analysis in Telegram and compare API-based collection, relational databases, graph databases, and open-data automation. Their prototype uses graph-oriented modeling to identify relationships between information channels [9]. This is directly relevant to ScamTracer's cross-conversation graph. Although the target platform differs, the analytical abstraction is transferable: messages become events, channels or participants become entities, and shared links or textual similarity become edges.

## III. Research Gap and Design Objectives

The reviewed literature provides strong individual solutions but leaves a systems-integration gap. Digital forensics emphasizes evidence; conversational studies emphasize language; fraud research emphasizes classification and imbalance; network research emphasizes graph structure; and honeypot research emphasizes controlled engagement. ScamTracer combines these elements into a single investigative pipeline.

The first gap is temporal integration. Existing message-level classifiers may produce a label for an individual message without representing how the interaction arrived at that message. ScamTracer models the conversation as a time-ordered sequence and computes features over sliding windows and stage hypotheses.

The second gap is cross-modal evidence fusion. A URL, phone number, payment ID, message pattern, and timestamp are heterogeneous objects. Treating them as independent columns loses relationships. ScamTracer preserves them as typed entities and events before projecting them into a graph.

The third gap is the distinction between detection and investigation. A classifier answers whether an observation resembles known fraudulent behavior. An investigator also needs to know what infrastructure is shared, when behavior changed, which entities recur, and what evidence supports the conclusion. ScamTracer therefore produces both a risk score and an evidence graph.

The fourth gap is privacy-aware engagement. Automated scambaiting introduces a new attack surface: the AI system itself might disclose information or make the conversation more dangerous. The engagement engine must consequently operate behind hard privacy constraints and explicit user authorization.

The fifth gap is reproducible analytics. A research system should make its transformation and feature-engineering steps inspectable. An R-centered implementation can keep data-cleaning scripts, feature tables, model objects, plots, statistical tests, and reports within a reproducible workflow.

The design objectives are therefore: (1) detect conversational risk; (2) characterize behavioral evolution; (3) extract technical and financial indicators; (4) connect related cases; (5) preserve forensic provenance; (6) minimize sensitive-data exposure; and (7) evaluate every analytical layer with measurable criteria.

## IV. Threat Model and Ethical Boundaries

ScamTracer assumes an adversary capable of adaptive social engineering. The adversary may use multiple accounts, rotate phone numbers, reuse payment identifiers, host several domains, alter wording, and migrate between communication channels. The adversary may also intentionally test whether a target is automated.

The framework does not assume that every suspicious participant is malicious, nor that every shared entity belongs to one actor. A shared URL can arise from an advertising service, a legitimate redirector, or common infrastructure. A linguistic similarity can result from copied templates rather

than coordinated authorship. A phone number can be reassigned. Consequently, graph edges are treated as indicators of association rather than proof of identity.

Privacy is a core design constraint. The system should operate on a dedicated investigation account or synthetic persona rather than a user's personal account. If live user interactions are used for research, explicit consent, data minimization, anonymization, and access control are required. Hossain et al. demonstrate a related principle by using federated learning to avoid centralized raw-data collection and by adding privacy safeguards around AI-generated responses [6], [26], [27].

The framework also distinguishes three evidence classes. *Observed evidence* includes directly captured message text, timestamps, URLs, identifiers, and network observations. *Derived evidence* includes counts, ratios, clusters, embeddings, and graph metrics computed from observed evidence. *Model inference* includes risk scores, classifications, and anomaly predictions. Reports should preserve these classes separately so that a model output is never mistaken for a directly observed fact.

## V. System Architecture

The complete system consists of six layers: acquisition, normalization, conversational analysis, behavioral modeling, graph intelligence, and forensic reporting.

### A. Acquisition Layer

The acquisition layer receives authorized messages and metadata through a controlled messaging interface. For WhatsApp-oriented deployment, the communication connector is separated from the analytical engine. The connector is responsible for message transport and collection; R is responsible for analytical processing. This separation limits the permissions required by the analytical component and makes the architecture easier to test.

Each event is assigned an internal identifier. The event record contains conversation ID, participant role, timestamp, message type, normalized text, raw-text hash, extracted entities, media indicators, and collection metadata. Raw content can be encrypted or stored separately from analytical features.

### B. Normalization Layer

Normalization converts heterogeneous messages into a common schema. Text normalization may include Unicode normalization, whitespace normalization, URL parsing, phone-number canonicalization, currency extraction, and language identification. Importantly, normalization should preserve the original text hash and should not destroy features such as capitalization, punctuation, repeated characters, or emoji patterns because these may contribute to classification.

A dual representation is therefore recommended: a forensic representation that preserves the source content and a modeling representation that is sanitized for statistical analysis. Verma and Sadana similarly use parallel textual and metadata feature streams for SMS classification, retaining handcrafted signals such as length, URLs, punctuation, and other message attributes alongside bag-of-words features [14].

### C. Conversational Analysis Layer

The conversational engine maintains dialogue state, participant roles, recent context, and a synthetic persona profile. Before generating a response, it checks whether engagement is authorized and whether the conversation has crossed a predefined risk threshold. Candidate responses are generated under constraints and passed through PII and safety filters.

A response utility can be represented as

$$U(g) = \alpha E(g) - \beta P(g) - \gamma H(g), \tag{1}$$

where $E(g)$ is expected engagement, $P(g)$ is privacy exposure risk, and $H(g)$ is harm or escalation risk. The response is transmitted only if it satisfies hard safety constraints. This formulation follows the general separation of engagement and risk control described in AI-in-the-loop scam-baiting research [6], [25].

### D. Behavioral Analysis Layer

The behavioral layer transforms event sequences into features. Features include message frequency, inter-arrival time, response latency, number of URLs, number of payment identifiers, entity repetition, conversation length, media frequency, lexical diversity, sentiment-related indicators, urgency markers, and stage-transition features.

For participant $i$ and time window $w$, message rate can be defined as

$$r_{i,w} = \frac{N_{i,w}}{\Delta t_w}, \tag{2}$$

where $N_{i,w}$ is the number of messages and $\Delta t_w$ is the window duration. Sudden changes in $r_{i,w}$ can become anomaly features without assuming that high activity is inherently malicious.

### E. Graph Intelligence Layer

The graph layer represents entities as typed nodes and interactions as typed edges. Candidate node types include conversation, account, phone number, URL, domain, payment identifier, cryptocurrency wallet, organization name, and message cluster. Edge types include `MENTIONED extunderscore IN`, `SHARED extunderscore WITH`, `LINKS extunderscore TO`, `SENT extunderscore BY`, and `CO extunderscore OCCURS_WITH`.

The graph can be represented as $G = (V, E, X)$, where $V$ is the node set, $E$ the relationship set, and $X$ the feature matrix. Degree, weighted degree, betweenness, PageRank-like centrality, community membership, and temporal recurrence can then be calculated. Graph methods are especially useful when several weak indicators combine into a stronger structural pattern [10], [11], [13], [55].

## VI. Stage-Aware Conversational Feature Engineering

A central component of ScamTracer is the conversion of raw conversation history into stage-aware features. The five-stage model from Li et al. provides a conceptual template:

Trust Cultivation, Risk Fabrication, Persuasive Pitching, Compliance Induction, and Repeated Exploitation [7]. These labels should initially be treated as analytical hypotheses rather than immutable ground truth.

During Trust Cultivation, the system may observe greetings, personal questions, affiliation language, shared interests, and low-pressure conversation. During Risk Fabrication, urgency, fear, scarcity, account problems, or fabricated emergencies may increase. Persuasive Pitching can introduce returns, rewards, authority cues, investment narratives, or exclusive opportunities. Compliance Induction can contain explicit requests for money, credentials, codes, or links. Repeated Exploitation can contain follow-up demands, new fees, recovery narratives, or repeated attempts after an earlier payment.

For each stage candidate $s$, a feature vector can be written as

$$X_s = [L_s, B_s, E_s, T_s, N_s], \tag{3}$$

where $L_s$ represents linguistic features, $B_s$ behavioral features, $E_s$ extracted entities, $T_s$ temporal features, and $N_s$ network or infrastructure indicators.

The linguistic component can include LIWC-style categories such as cognitive processes, discrepancy, certainty, tentativeness, affiliation, power, achievement, money, and risk [32]–[34]. The system should not treat these features as psychological diagnoses. Their purpose is to quantify communication patterns associated with scam narratives.

Temporal features are equally important. A scammer who mentions money in the first message differs from a scammer who establishes a long interaction before introducing a financial request. Features can therefore include time-to-first-payment-request, time-to-first-URL, number of escalation events, change in urgency density, and the ratio of financial terms before and after a stage transition.

## VII. Statistical Analysis and R-Centered Implementation

R provides a natural analytical environment for the proposed framework. The initial data frame can contain one row per message and columns for conversation ID, sender role, timestamp, text, message type, URL count, phone count, payment-ID count, word count, and derived linguistic categories.

The tidy-data representation enables grouping by conversation, participant, stage, and time window. Functions from `dplyr` can perform aggregation and transformation, while `tidyr` can reshape nested event records. String processing packages can support entity extraction and regular expressions. Statistical testing can then be conducted over feature distributions.

For example, a comparison between scammer and victim linguistic behavior can be formulated as a two-group test, while stage-dependent behavior can be evaluated using analysis of variance or mixed-effects models. Li et al. use linear mixed-effects modeling to account for nested conversation data, and Bates et al. provide the widely used `lme4` implementation [7], [36]. A model of a feature $y$ can be expressed as

$$y_{ijk} = \beta_0 + \beta_1 Role_i + \beta_2 Stage_j + u_k + \epsilon_{ijk}, \tag{4}$$

where $u_k$ represents a conversation-level random effect.

The R implementation can also calculate correlation matrices between linguistic and behavioral variables. Correlation should be interpreted as association rather than causation. Multiple-testing correction should be applied when many features are tested simultaneously.

For classification, the analytical dataset can be divided into training, validation, and test partitions at the conversation level. Splitting individual messages from the same conversation across training and test sets would create leakage because neighboring messages share vocabulary, entities, and context. Conversation-level splitting is therefore required for credible evaluation.

## VIII. Machine-Learning Model Design

The classifier layer should combine supervised and unsupervised methods. Supervised models can include logistic regression, random forests, support vector machines, gradient-boosted trees, and XGBoost. Unsupervised methods can include k-means, hierarchical clustering, DBSCAN, isolation-based anomaly detection, and one-class SVM.

A hybrid risk score can be expressed as

$$R = \lambda_1 R_{text} + \lambda_2 R_{behavior} + \lambda_3 R_{entity} + \lambda_4 R_{graph}, \tag{5}$$

where each component is normalized to a common scale. The weights should be learned or validated rather than selected only because they produce visually plausible results.

Class imbalance requires special handling. Possible methods include class weighting, stratified sampling, threshold tuning, and resampling. Hashemi et al. demonstrate weight tuning and ensemble combinations for imbalanced banking fraud, while Irofti et al. show why one-class approaches can be useful when anomalous examples are scarce [4], [8], [41], [42].

The decision threshold should be selected according to the intended operational cost. A low threshold can generate more review candidates but may increase false positives. A high threshold may reduce review load but miss emerging scams. Therefore, the paper should report threshold-dependent precision and recall rather than present one universal threshold as optimal.

## IX. Entity Extraction and Resolution

ScamTracer focuses on entities that can support investigation without requiring identity attribution. Phone numbers can be normalized using country-code rules; URLs can be decomposed into scheme, domain, path, query parameters, and registrable domain; payment identifiers can be classified by syntax; and cryptocurrency addresses can be detected using network-specific patterns.

Entity resolution is challenging because the same entity can appear in multiple textual forms. URL shorteners, tracking parameters, capitalization, whitespace, and punctuation can produce different strings that refer to the same destination. Canonicalization should therefore be applied before graph insertion, while the original form is retained for evidence.

A confidence value should accompany every extracted entity. A regular expression that matches a phone-number pattern

does not prove that the string is a valid or active phone number. Similarly, a name-like sequence is not sufficient for identity attribution. The graph should therefore store extraction method and confidence alongside the value.

## X. Graph Construction and Community Analysis

The graph is constructed incrementally as new conversations arrive. Each newly observed entity is matched against existing canonical entities. If a match exists, the system creates an additional temporal edge rather than a duplicate node. This allows the graph to preserve recurrence.

Community detection can reveal groups of conversations connected through shared URLs, phone numbers, payment identifiers, or behavioral signatures. Centrality measures can identify entities that occur across many cases. However, high centrality is not itself evidence of maliciousness. Shared public services can also become central nodes. Analysts must therefore inspect the edge types and provenance behind every graph pattern.

Temporal graphs provide an additional dimension. Let $e_{uv}(t)$ denote an interaction between entities $u$ and $v$ at time $t$. Repeated edges in short intervals can be represented by a temporal intensity function

$$I(u, v) = \sum_{t} w_t e_{uv}(t), \tag{6}$$

where $w_t$ weights recent observations. Such a measure can support investigation of rapidly changing campaigns while avoiding permanent assumptions about relationships.

Wang and Zhu's heterogeneous network approach is particularly relevant because ScamTracer contains multiple node types and relationship semantics [11], [47], [48], [52]. A homogeneous graph that connects every object through one generic edge would lose this information. Typed relationships allow an analyst to distinguish a URL shared in a conversation from a URL redirecting to a domain, or a phone number appearing in a message from an account using that number.

## XI. Forensic Evidence and Explainability

A core output of ScamTracer is a forensic-ready case bundle. The bundle contains a case identifier, acquisition metadata, conversation timeline, extracted indicators, model outputs, graph snapshot, analytical transformations, and evidence hashes.

Every model result should be traceable to the features that influenced it. Tree-based models can provide feature importance, while linear models can expose coefficients. For graph results, the report should display the subgraph responsible for the alert and list the exact observations supporting each edge.

The evidence model should preserve a distinction between fact and interpretation. For example, a report can state that two conversations contained the same URL. It can then state that the graph model assigned a high structural-association score to the two cases. It should not automatically state that both conversations were operated by the same person. This conservative reporting approach is consistent with forensic principles and avoids overclaiming from probabilistic outputs [5], [16], [17].

A reproducibility manifest can include software version, R version, package versions, feature-extraction configuration, model hyperparameters, training-data hash, and execution timestamp. This turns the analytical workflow into an auditable research artifact.

## XII. Evaluation Methodology

The proposed system should be evaluated at multiple levels instead of using a single end-to-end accuracy value.

### A. Detection Evaluation

Conversation-level detection should report precision, recall, F1, ROC-AUC, PR-AUC, specificity, and Matthews correlation coefficient. PR-AUC is especially important under severe class imbalance. Confidence calibration can be evaluated using reliability curves and Brier score.

### B. Stage Detection Evaluation

Stage classification should be evaluated using macro-F1 because some stages may have fewer examples. Confusion matrices can show whether Trust Cultivation and Risk Fabrication are systematically confused or whether Compliance Induction is detected reliably.

### C. Entity Extraction Evaluation

Entity extraction should use precision, recall, and F1 for each entity type. Exact-match and partial-match evaluation can be separated for URLs and names. False positives should be categorized because an overly aggressive extractor can pollute the graph.

### D. Graph Evaluation

Graph evaluation should measure community purity on datasets with known campaign relationships, link-prediction performance for held-out relationships, and analyst usefulness. Since real fraud-ring ground truth is difficult to obtain, synthetic and semi-synthetic benchmark graphs can complement real-world data.

### E. Engagement Evaluation

Conversational engagement should not be measured solely by conversation length. Metrics can include number of turns, scammer response rate, information yield, safety violations, PII leakage rate, and time until disengagement. Hossain et al. use engagement, relevance, and PII leakage as part of their evaluation, providing a useful model for ScamTracer's engagement tests [6].

### F. Ablation Studies

Ablation experiments should remove one evidence source at a time: text only, behavior only, entities only, graph only, text plus behavior, and full fusion. This determines whether graph features provide incremental information and whether the conversational stage representation contributes beyond message-level features.

## XIII. Illustrative Investigation Workflow

Consider a synthetic conversation in which an unknown contact begins with a friendly greeting, gradually discusses a financial opportunity, sends a shortened URL, requests a small verification payment, and later claims that an additional fee is required. The system first stores the messages and timestamps. The normalization layer extracts the URL and payment request. The linguistic layer identifies changes in money, reward, urgency, and directive language. The behavioral layer measures the time between the initial contact and financial request.

The conversation is then represented as a sequence rather than a single record. If the same URL appears in another case, a shared entity node is created. If a payment identifier appears in several conversations, the graph creates multiple temporal edges. Community detection can then identify a connected group of cases for analyst review.

The final report might contain a timeline showing the transition from trust-oriented language to financial persuasion, an entity table listing observed indicators, and a graph showing relationships among conversations and infrastructure. The model may assign a high risk score, but the report retains the underlying observations so that the analyst can independently inspect them.

This example illustrates why the system is more than a classifier. A classifier could identify the financial request. The proposed architecture additionally explains how the request emerged, what entities were involved, whether similar entities occurred elsewhere, and which observations produced the alert.

## XIV. Security of the ScamTracer System

Because ScamTracer interacts with adversaries, the system itself must be treated as an attack surface. Prompt injection is one important threat. A scammer may attempt to manipulate the LLM by sending instructions such as requests to reveal the system prompt or disclose stored information. The engagement engine should therefore separate untrusted conversation text from system instructions and enforce output policies outside the LLM.

Data exfiltration is another threat. If conversation data is sent to external services, the privacy boundary changes. A deployment should therefore minimize transmitted content, redact identifiers when full values are unnecessary, and maintain explicit records of third-party processing.

Graph poisoning is also possible. An attacker can intentionally introduce large numbers of fake entities or relationships to distort community structure. Mitigations include edge provenance, confidence weighting, temporal decay, duplicate detection, and analyst review of high-impact nodes.

Model drift is inevitable because scam language and infrastructure change. Periodic retraining should therefore be accompanied by drift monitoring. Changes in feature distributions, precision, recall, and false-positive categories should be tracked over time.

## XV. Limitations

The proposed architecture has several limitations. First, conversational data can be difficult to obtain ethically and legally. Public scam datasets may not represent private messaging behavior. Second, stage labels can be subjective, particularly when a conversation skips or repeats stages. Third, entity extraction errors can propagate into the graph. Fourth, graph connectivity can be caused by legitimate shared infrastructure. Fifth, machine-learning models can learn dataset-specific artifacts rather than general scam behavior.

A further limitation is adversarial adaptation. Once scammers recognize common detection signals, they can alter wording, delay payment requests, rotate infrastructure, or use benign-looking content. The system must therefore treat its feature set as adaptive rather than permanent.

The LLM engagement layer introduces additional uncertainty. A generated response that appears plausible to one scammer may appear artificial to another. More importantly, optimizing for engagement can conflict with safety. The system should therefore prioritize hard privacy and harm constraints over conversation length. Engagement is useful only when it can be maintained without creating unacceptable risk.

## XVI. Future Research Directions

Future work can extend ScamTracer in several directions. First, multimodal analysis can incorporate images, audio, and documents while preserving modality-specific provenance. Second, voice scams can be modeled through speech-to-text, acoustic features, and call metadata. Third, graph neural networks can learn representations over heterogeneous temporal graphs, although their added complexity should be justified against interpretable baselines.

A second direction is privacy-preserving collaborative learning. Federated learning can allow multiple organizations or user devices to contribute model updates without centralizing raw conversations. Secure aggregation and differential privacy can reduce leakage risks, while personalized federated methods can address distribution differences between users [6], [26]–[28], [65].

Third, the R package proposed for the analytical layer can expose standardized functions for cleaning, entity extraction, behavioral feature generation, risk modeling, graph conversion, and report creation. This would make the research pipeline reusable for future scam datasets.

Fourth, evaluation should expand beyond binary scam detection. Research questions can examine early-warning performance, time-to-detection, stage transition detection, infrastructure recurrence, analyst workload, and evidence quality. These metrics better represent an investigative intelligence system than classification accuracy alone.

## XVII. Discussion

The literature indicates that scam detection is moving from isolated indicators toward contextual and relational analysis.

The ScamTracer architecture follows this direction by combining communication, behavior, language, infrastructure, and graph structure.

The strongest methodological implication is that no individual evidence source should be treated as sufficient. Text can be ambiguous; behavior can be unusual but legitimate; URLs can be shared by unrelated actors; and graph connections can arise from common services. Evidence fusion can improve context, but it also increases the risk of overinterpretation. Therefore, ScamTracer should produce ranked investigative leads with explicit provenance rather than automated declarations of guilt or identity.

The second implication is that interaction itself can become a source of evidence. Traditional systems passively inspect messages, while honeypot and scambaiting approaches create controlled opportunities to observe adaptive behavior. This changes the detection problem from classification of a fixed artifact to analysis of a dynamic adversarial process.

The third implication is that R can support more than academic visualization. An R-centered implementation can make the entire analytical lifecycle reproducible: data preparation, statistical testing, machine learning, graph construction, evaluation, and reporting. The use of R also encourages explicit tabular representations that can be audited and inspected before complex models are applied.

Finally, the proposed architecture highlights an important distinction between intelligence and attribution. ScamTracer can identify recurring indicators, suspicious behavioral patterns, and network associations. These outputs can support investigation, but attribution requires additional evidence and legal or investigative procedures outside the model. Preserving this boundary is essential for a system intended for cybercrime research.

## XVIII. Conclusion

This extended manuscript develops ScamTracer AI from a conceptual three-page draft into a detailed research architecture for proactive scam intelligence. The design integrates conversational engagement, linguistic stage analysis, behavioral anomaly detection, imbalanced fraud classification, entity extraction, graph-based intelligence, and digital-forensic reporting. The literature reviewed across the fifteen bibliography entries in the draft supports each component: WhatsApp network forensics motivates communication-level evidence; computer-forensics surveys motivate provenance and preservation; honeypots motivate controlled engagement; conversational scam studies motivate stage-aware linguistic analysis; fraud-classification studies motivate imbalance-aware evaluation; anomaly-detection research motivates unsupervised screening; and graph studies motivate cross-case relationship analysis.

The proposed system is deliberately framed as an evidence-generation and decision-support framework rather than an autonomous attribution mechanism. Its outputs should remain traceable to observations, derived features, and model inferences. Privacy, safety, provenance, and evaluation are therefore architectural requirements rather than optional additions. Future implementation and experiments can validate the proposed pipeline using authorized datasets and controlled test environments, with R serving as the central analytical layer and graph technologies providing relational intelligence.

TABLE I
Core Message-Level Data Model

| Field | Purpose |
|---|---|
| conversation_id | Links messages belonging to one interaction. |
| event_id | Unique immutable identifier for an event. |
| timestamp | Preserves temporal order and supports interval analysis. |
| role | Indicates the analytical role assigned to the participant. |
| text_hash | Integrity reference for the source text. |
| message_type | Text, image, document, link, or other observed type. |
| url_count | Number of detected URL strings. |
| payment_count | Number of payment-related identifiers detected. |
| word_count | Basic lexical size feature. |
| urgency_density | Density of urgency-related terms in a window. |
| stage | Stage hypothesis assigned by the analytical layer. |
| risk_score | Model-derived risk value with model version. |

## XIX. Detailed Data Model

A practical implementation requires a stable data model before any machine-learning model is trained. The proposed representation separates raw events, normalized events, extracted entities, behavioral windows, model predictions, and graph relationships. This separation prevents analytical transformations from overwriting the source evidence and allows the same raw conversation to be reprocessed with a new feature extractor.

### A. Message-Level Record

Each message is represented by a record containing at least the fields shown in Table I. The fields are deliberately divided into observed and derived attributes. Observed fields originate from the communication interface, whereas derived fields are produced by deterministic preprocessing or analytical models.

A separate entity table stores canonical entities. It can contain entity ID, entity type, normalized value, first-seen time, last-seen time, extraction method, extraction confidence, and source-event count. A third relationship table stores source entity, target entity, relationship type, timestamp, weight, and provenance. This relational representation can be exported directly into a graph database or analyzed with R network packages before graph persistence.

### B. Temporal Windows

A conversation should be represented at multiple temporal resolutions. Message-level features capture local behavior, while five-minute, thirty-minute, daily, and conversation-level windows capture broader patterns. Let $x_t$ be a feature measured at time $t$. A rolling mean is

$$\bar{x}_t = \frac{1}{k} \sum_{i=0}^{k-1} x_{t-i}, \tag{7}$$

where $k$ is the number of observations in the window. A rolling standard deviation can detect sudden behavioral changes. Such

transformations are useful for identifying escalation without assigning a fixed semantic label to every individual message.

The windowing strategy should be determined from the dataset rather than chosen only for convenience. Very short windows may be noisy, whereas very long windows can hide rapid changes. Multi-scale features allow the classifier to retain both local and global behavior.

## XX. Feature Catalogue for ScamTracer

The feature space can be organized into six groups: lexical, conversational, temporal, entity, media, and graph features. This organization allows ablation experiments to determine which evidence types contribute to detection.

### A. Lexical Features

Lexical features include token counts, character counts, average word length, vocabulary size, repeated-token ratio, uppercase ratio, punctuation density, digit density, URL density, and n-gram frequencies. Verma and Sadana demonstrate that simple metadata features can complement bag-of-words representations in spam classification [14], [59], [60]. For ScamTracer, lexical features are intentionally lightweight so that they can be computed in real time.

Psycholinguistic categories add a more interpretable layer. The system can calculate frequencies of money, reward, risk, power, achievement, affiliation, certainty, tentativeness, cognitive-process, and emotional terms. These categories are motivated by LIWC-based deception research and the COLD analysis of scam conversations [7], [32]–[34].

### B. Conversation Features

Conversation-level features capture interaction structure. These include turn count, average response delay, median response delay, ratio of sender to receiver messages, number of unanswered questions, number of repeated requests, number of financial requests, and number of escalation events. The ratio

$$q = \frac{N_{request}}{N_{message} + \epsilon} \tag{8}$$

can represent the density of requests without depending directly on conversation length.

Another useful feature is persistence after refusal. If a participant receives a refusal and immediately repeats the same request, the event can be represented as a refusal-response pair. Repeated patterns can then be aggregated into a persistence score. This does not establish malicious intent, but it provides a measurable behavioral signal for model training.

### C. Temporal Features

Temporal features include inter-arrival times, burstiness, time-of-day distribution, weekday distribution, escalation delay, and entity recurrence intervals. Burstiness can be approximated by comparing the standard deviation and mean of inter-arrival times. A participant whose messages arrive at highly irregular intervals may have a different behavioral profile from a participant who communicates steadily.

Time features can also support campaign correlation. If the same domain appears across many conversations within a short interval, the graph can increase the temporal weight of the associated edges. Temporal weighting should decay gradually so that old relationships remain visible without dominating current activity.

### D. Entity Features

Entity features include counts and recurrence of phone numbers, URLs, domains, payment identifiers, email addresses, organization names, and cryptocurrency addresses. The number of unique entities and the reuse ratio are particularly useful. For example,

$$ReuseRatio = 1 - \frac{|E_{unique}|}{|E_{mentions}| + \epsilon}. \tag{9}$$

A high value means that a small number of entities are being reused frequently. This may indicate campaign infrastructure, although legitimate services can also exhibit high reuse.

### E. Media and Structural Features

Media features include whether an interaction contains images, documents, audio, or video; the number of media objects; file-type distribution; and whether media is introduced immediately before a request. OCR can optionally convert image text into a derived representation. Because OCR can produce errors, the original image hash and extraction confidence should remain attached to the derived text.

### F. Graph Features

Graph features include node degree, weighted degree, PageRank-like centrality, betweenness, clustering coefficient, community ID, community size, temporal edge count, and number of distinct entity types connected to a node. These features allow the classifier to distinguish an isolated conversation from a conversation embedded in a broader network.

## XXI. Cross-Source Synthesis of the Fifteen Draft Papers

The fifteen entries in the draft can be mapped into an evidence-to-method matrix. This matrix makes explicit how references extracted from each source contribute to the expanded design.

The table demonstrates that the expanded manuscript does not treat the fifteen papers as a flat bibliography. Instead, at least five references or reference-derived methods from each source family are incorporated into the architecture. For example, the digital-forensics source contributes behavioral evidence analysis, privacy considerations, and network-forensics concepts; the graph sources contribute node embeddings, heterogeneous relationships, anomaly detection, and community analysis; and the linguistic source contributes segmentation, LIWC categories, deception studies, and mixed-effects modeling.

TABLE II
LITERATURE-TO-METHOD MAPPING USED IN THE EXTENDED MANUSCRIPT

| Source paper | Reference themes extracted | ScamTracer use in this manuscript |
|---|---|---|
| Kao et al. | WhatsApp forensics, packet filtering, network indicators | Communication-level evidence, network metadata, forensic acquisition. |
| Liu et al. | Technical-support scams, search abuse, spam value chain, scam infrastructure | Campaign measurement and infrastructure-aware detection. |
| Uchibori et al. | Honeypots, Ethereum security, clustering, attack observation | Controlled engagement and behavioral clustering. |
| Hashemi et al. | Class weighting, Bayesian optimization, LightGBM, XGBoost, CatBoost | Imbalanced classification and threshold-aware evaluation. |
| Javed et al. | Computer forensics, behavioral evidence, privacy, network and email forensics | Evidence preservation, provenance, and multi-domain forensic design. |
| Hossain et al. | AI-in-the-loop, scambaiting, federated learning, differential privacy, safety filtering | Privacy-aware conversational engagement. |
| Li et al. | COLD framework, LIWC, deception, mixed-effects modeling, stage segmentation | Stage-aware linguistic and statistical analysis. |
| Irofti et al. | One-class learning, graph anomaly detection, clustering, attributed networks | Unsupervised anomaly screening and graph anomalies. |
| Chornyi and Stopochkina | Telegram APIs, information-flow graphs, graph databases | Cross-channel graph construction and information-flow analysis. |
| Modepalli | Community detection, centrality, network construction, fraud rings | Graph intelligence and campaign-level analysis. |
| Wang and Zhu | Co-occurrence graphs, knowledge graphs, heterogeneous embeddings, node2vec | Typed entity relationships and representation learning. |
| Liu et al. | Log features, K-prototype clustering, k-NN, refinement | Multi-stage anomaly pipeline and behavioral feature extraction. |
| Kurshan et al. | Graph computing, financial crime, graph neural networks, implementation constraints | Graph architecture and operational considerations. |
| Verma and Sadana | Bag-of-words, metadata, Naive Bayes, SVM, adversarial robustness | Hybrid text-plus-metadata features and lightweight deployment. |
| Hashemi et al. (duplicate draft entry) | Same banking-fraud study as entry [4] | Incorporated once in the analytical synthesis; duplicate bibliography entry retained in the source traceability note. |

## XXII. EXPERIMENTAL DATASET STRATEGY

A robust evaluation requires multiple datasets because no single collection can represent all scam types. The first dataset should contain labeled scam and legitimate conversations. The second should contain infrastructure indicators such as URLs and domains. The third should contain synthetic conversations generated under controlled scenarios. A fourth dataset can be constructed from temporal graph simulations to test community detection and link recurrence.

The real-world dataset should be partitioned by conversation, not by message. If messages from one conversation appear in both training and testing, the classifier can learn conversation-specific vocabulary and entities, creating an artificially optimistic result. Grouped cross-validation is therefore required. Where multiple conversations are linked by the same campaign, campaign-level splitting should also be considered to measure generalization to unseen campaigns.

The synthetic dataset should vary scam type, language style, conversation length, stage transitions, entity reuse, and adversarial behavior. Synthetic data should not replace real-world evaluation; it should isolate individual mechanisms that are difficult to control in natural data.

### A. Data Quality Checks

Before training, the dataset should be inspected for duplicate conversations, repeated messages, missing timestamps, inconsistent time zones, corrupted Unicode, mislabeled roles, and leaked identifiers. The proportion of missing values should be reported for each feature family. Feature distributions should be visualized before and after preprocessing so that normalization does not unintentionally remove discriminative signals.

### B. Label Quality

Scam labels should ideally come from multiple independent sources. If labels are manually assigned, inter-rater agreement should be reported. The segmentation methodology in Li et al. demonstrates the usefulness of explicit coding protocols and agreement measurement for conversational stages [7], [36]. Disagreements should be preserved during the research process rather than silently converted into one label.

## XXIII. BASELINE AND COMPARATIVE MODELS

The first baseline should be a keyword or rule-based detector. It is simple and interpretable but expected to be vulnerable to paraphrasing and adversarial wording. The second baseline should be a bag-of-words logistic regression model. The third should be a Naive Bayes classifier, reflecting the lightweight

text classification literature. The fourth should be an SVM using text features, motivated by Verma and Sadana. The fifth should be a gradient-boosted tree using hybrid textual and metadata features. The sixth should be a graph-only anomaly detector.

The proposed fusion model should then combine text, behavior, entity, and graph features. The comparison should be made under the same conversation-level train/test split. This prevents improvements from being attributed merely to easier data partitions.

The study should also compare early-warning detection against full-conversation detection. Early-warning evaluation asks how much evidence is needed before the system flags an interaction. This is important because a detector that becomes accurate only after a victim has already paid is less useful for prevention than one that identifies risk during the early stages of manipulation.

## XXIV. Threshold and Calibration Analysis

A risk score without calibration can be misleading. If the model outputs 0.8, the number should have an interpretable relationship with observed risk in the validation data. Calibration curves can compare predicted probability bins with empirical event frequencies. Platt scaling or isotonic regression can be evaluated where appropriate.

Operational thresholds can then be defined as policy parameters. For example, a low threshold can trigger passive logging, a medium threshold can request analyst review, and a high threshold can activate a controlled engagement mechanism. These are system states rather than universal claims about maliciousness.

Thresholds should also be stress-tested under changing scam prevalence. A detector evaluated on a balanced benchmark may behave differently when deployed in a population where legitimate messages dominate. Sensitivity analysis should therefore vary the base rate and measure expected alert volume.

## XXV. Adversarial and Robustness Testing

ScamTracer should be tested against adversarial transformations. Text-level transformations include synonym substitution, spelling variation, punctuation manipulation, code-switching, transliteration, and intentional obfuscation of URLs or phone numbers. Infrastructure-level transformations include domain rotation, URL shorteners, and changes in payment identifiers.

Verma and Sadana explicitly evaluate adversarial changes and show the value of combining lexical and metadata features [14]. This supports an important design principle: different feature families should fail differently. If a lexical attack reduces word-level similarity, metadata and behavioral signals may still remain available.

The graph layer also requires adversarial testing. An attacker may create decoy entities to dilute community structure. The evaluation can inject controlled noisy nodes and edges and measure whether the intended community remains recoverable. Robust graph analysis should use provenance and confidence rather than treating every edge as equally reliable.

## XXVI. Operational Deployment Architecture

A deployment can be separated into a communication connector, analytical service, feature store, graph store, and reporting interface. The connector receives authorized events. The analytical service performs normalization and model inference. The feature store maintains derived numerical data. The graph store maintains entities and relationships. The reporting interface provides timelines, tables, network views, and evidence provenance.

An R-based implementation can expose the analytical layer through a service interface. R scripts or an R package can implement deterministic transformations, statistical analysis, model training, and report generation. A Shiny dashboard can provide interactive exploration, while a Plumber API can expose scoring endpoints where required. This arrangement also permits batch experiments without requiring the dashboard to remain online.

The system should maintain model-version identifiers. Every risk score must therefore be associated with the model, feature configuration, and training-data version used to produce it. When a model is retrained, historical scores should not be silently overwritten. This is essential for reproducibility and longitudinal comparison.

## XXVII. Human-in-the-Loop Investigation

Although ScamTracer automates large parts of collection and analysis, human review remains important. An analyst should be able to inspect the messages behind a feature, expand the graph edge that triggered a connection, view the timeline around a risk transition, and compare the current case with previous cases.

Human review should also be used to improve the training dataset. Analysts can mark false positives, missed indicators, incorrect entities, and misleading graph connections. These corrections can become new labeled examples after appropriate quality control.

This feedback loop resembles interactive anomaly-detection research in which human experts provide limited labels to improve anomaly discovery [56]. The important distinction is that analyst feedback should be recorded as a separate evidence source and should not automatically be treated as objective ground truth.

## XXVIII. Research Reproducibility Plan

Reproducibility requires more than publishing a final model. ScamTracer should maintain a versioned data dictionary, feature specification, preprocessing configuration, training split, model configuration, evaluation script, and report template. Random seeds should be recorded for stochastic algorithms.

A recommended experiment directory contains: `data/` for controlled datasets, `R/` for analytical scripts, `models/` for serialized model objects, `graphs/` for graph exports, `reports/` for generated figures and tables, and `config/` for experiment parameters. Sensitive source conversations should never be committed to a public repository. Only anonymized or synthetic samples should be distributed.

Each experiment should produce a machine-readable summary containing dataset version, number of conversations, number of messages, positive rate, feature count, model name, hyperparameters, threshold, precision, recall, F1, PR-AUC, ROC-AUC, calibration metric, and runtime. This creates an auditable comparison between experiments.

## XXIX. Expected Research Contributions

The proposed study has five research contributions. First, it presents a unified architecture for conversational scam intelligence that joins engagement, behavioral analysis, and graph investigation. Second, it introduces a stage-aware feature representation that treats scam conversations as evolving interactions. Third, it provides a typed graph model for linking conversational entities and infrastructure. Fourth, it proposes an R-centered reproducible analytical implementation. Fifth, it defines an evaluation framework that measures detection, early warning, entity extraction, graph usefulness, engagement safety, and privacy rather than relying on one classification metric.

These contributions are deliberately framed as research objectives until validated experimentally. The extended manuscript does not claim that the proposed architecture has already achieved a particular accuracy or uncovered a real fraud ring. Such conclusions require controlled experiments and appropriately labeled datasets.

## XXX. Comparative Analysis of Existing Approaches

## REFERENCES


[1] D.-Y. Kao, E.-C. Chang, and F.-C. Tsai, "Extracting suspicious IP addresses from WhatsApp network traffic in cybercrime investigations," in *Proc. 21st Int. Conf. Advanced Communication Technology (ICACT)*, 2019, pp. 1108–1115, doi: 10.23919/ICACT.2019.8701941.

[2] J. Liu, P. Pun, P. Vadrevu, and R. Perdisci, "Understanding, measuring, and detecting modern technical support scams," in *Proc. IEEE 8th European Symp. Security and Privacy (EuroS&P)*, 2023, pp. 18–38, doi: 10.1109/EuroSP57164.2023.00011.

[3] H. Uchibori, K. Yoshioka, and K. Omote, "Honeypot method to lure attackers without holding crypto-assets," *IEEE Access*, vol. 12, pp. 16059–16071, 2024, doi: 10.1109/ACCESS.2024.3357785.

[4] S. K. Hashemi, S. L. Mirtaheri, and S. Greco, "Fraud detection in banking data by machine learning techniques," *IEEE Access*, vol. 11, pp. 3034–3043, 2023, doi: 10.1109/ACCESS.2022.3232287.

[5] A. R. Javed, W. Ahmed, M. Alazab, Z. Jalil, K. Kifayat, and T. R. Gadekallu, "A comprehensive survey on computer forensics: State-of-the-art, tools, techniques, challenges, and future directions," *IEEE Access*, vol. 10, pp. 11065–11089, 2022, doi: 10.1109/ACCESS.2022.3142508.

[6] I. Hossain, S. Puppala, S. Talukder, and M. J. Alam, "AI-in-the-loop: Privacy preserving real-time scam detection and conversational scam-baiting by leveraging LLMs and federated learning," arXiv:2509.05362, 2025/2026.

[7] D. Li, R. Zheng, X.-F. Liu, B. Ma, H. Sun, and L. C. Jiang, "Linguistic dynamics of online scam conversations: A multi-stage analysis based on the COLD framework," *Humanities and Social Sciences Communications*, vol. 13, Art. no. 698, 2026, doi: 10.1057/s41599-026-07052-y.

[8] P. Irofti, A. Patrascu, and A. Baltoiu, "Fraud detection in networks: State-of-the-art," arXiv:1910.11299, 2019.

[9] A. Chornyi and I. Stopochkina, "Graph-based analysis of information flows in Telegram for cybersecurity threat detection," *Cybersecurity: Education, Science, Technique*, no. 3, 2025, doi: 10.28925/2663-4023.2025.27.746.

[10] U. P. Modepalli, "Network analytics for identifying fraud rings and systemic risk," *Journal of Information Systems Engineering and Management*, vol. 10, no. 58s, pp. 616–624, 2025, doi: 10.52783/jisem.v10i58s.12641.

[11] C. Wang and H. Zhu, "Representing fine-grained co-occurrences for behavior-based fraud detection in online payment services," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 301–315, 2022, doi: 10.1109/TDSC.2020.2991872.

[12] Z. Liu, T. Qin, X. Guan, H. Jiang, and C. Wang, "An integrated method for anomaly detection from massive system logs," *IEEE Access*, vol. 6, pp. 30602–30611, 2018, doi: 10.1109/ACCESS.2018.2843336.

[13] E. Kurshan, H. Shen, and H. Yu, "Financial crime & fraud detection using graph computing: Application considerations & outlook," in *Proc. 2nd Int. Conf. Transdisciplinary AI (TransAI)*, 2020, pp. 125–130, doi: 10.1109/TransAI49837.2020.00029.

[14] A. R. Verma and S. Sadana, "Textual, non-textual, and hybrid feature engineering for SMS spam classification," *IEEE Access*, vol. 13, pp. 176901–176914, 2025, doi: 10.1109/ACCESS.2025.3620751.

[15] W. Ahmed, F. Shahzad, A. R. Javed, F. Iqbal, and L. Ali, "WhatsApp network forensics: Discovering the IP addresses of suspects," in *Proc. 11th IFIP Int. Conf. New Technol., Mobility Secur. (NTMS)*, 2021, pp. 1–7.

[16] N. A. Mutawa, J. Bryce, V. N. L. Franqueira, A. Marrington, and J. C. Read, "Behavioural digital forensics model: Embedding behavioural evidence analysis into the investigation of digital crimes," *Digital Investigation*, vol. 28, pp. 70–82, 2019.

[17] S. L. Garfinkel, "Digital forensics research: The next 10 years," *Digital Investigation*, vol. 7, pp. S64–S73, 2010.

[18] B. Carrier, *File System Forensic Analysis*. Reading, MA, USA: Addison-Wesley, 2005.

[19] S. Niksefat, P. Kaghazgaran, and B. Sadeghiyan, "Privacy issues in intrusion detection systems: A taxonomy, survey and future directions," *Computer Science Review*, vol. 25, pp. 69–78, 2017.

[20] M. Hina, M. Ali, A. R. Javed, F. Ghabban, L. A. Khan, and Z. Jalil, "SeFACED: Semantic-based forensic analysis and classification of e-mail data using deep learning," *IEEE Access*, vol. 9, pp. 98398–98411, 2021.

[21] S. K. Kim, Z. Ma, S. Murali, J. Mason, A. Miller, and M. Bailey, "Measuring Ethereum network peers," in *Proc. Internet Measurement Conf.*, 2018.

[22] C. F. Torres, M. Steichen, and R. State, "The art of the scam: Demystifying honeypots in Ethereum smart contracts," in *Proc. 28th USENIX Security Symp.*, 2019, pp. 1591–1607.

[23] W. Chen, X. Guo, Z. Chen, Z. Zheng, Y. Lu, and Y. Li, "Honeypot contract risk warning on Ethereum smart contracts," in *Proc. IEEE Int. Conf. Joint Cloud Comput.*, 2020, pp. 1–8.

[24] G. Münz, S. Li, and G. Carle, "Traffic anomaly detection using K-means clustering," in *Proc. GI/ITG Workshop MMBnet*, 2007.

[25] P. Bajaj and M. Edwards, "Automatic scam-baiting using ChatGPT," in *Proc. IEEE 22nd Int. Conf. Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2023, pp. 1941–1946.

[26] H. B. McMahan, E. Moore, D. Ramage, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.

[27] P. Kairouz, H. B. McMahan, B. Avent, et al., "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, nos. 1–2, pp. 1–210, 2021.

[28] J. Hayes, B. Balle, and S. Mahloujifar, "Bounding training data reconstruction in DP-SGD," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[29] G. D. Bond and A. Y. Lee, "Language of lies in prison: Linguistic classification of prisoners' truthful and deceptive natural language," *Applied Cognitive Psychology*, vol. 19, no. 3, pp. 313–329, 2005.

[30] E. Carter, "Confirm not command: Examining fraudsters' use of language to compel victim compliance in their own exploitation," *British Journal of Criminology*, vol. 63, no. 6, pp. 1405–1422, 2023.

[31] C. L. Toma and J. T. Hancock, "What lies beneath: The linguistic traces of deception in online dating profiles," *Journal of Communication*, 2012.

[32] M. L. Newman, J. W. Pennebaker, D. S. Berry, and J. M. Richards, "Lying words: Predicting deception from linguistic styles," *Personality and Social Psychology Bulletin*, vol. 29, no. 5, pp. 665–675, 2003.

[33] R. L. Boyd, A. Ashokkumar, S. Seraj, and J. W. Pennebaker, "The development and psychometric properties of LIWC-22," University of Texas at Austin, 2022.

[34] Y. R. Tausczik and J. W. Pennebaker, "The psychological meaning of words: LIWC and computerized text analysis methods," *Journal of Language and Social Psychology*, vol. 29, no. 1, pp. 24–54, 2010.

[35] D. M. Markowitz and J. T. Hancock, "Deception and context: The contextual organization of language and deception framework," 2019.

[36] D. M. Bates, M. Mächler, B. Bolker, and S. Walker, "Fitting linear mixed-effects models using lme4," *Journal of Statistical Software*, vol. 67, no. 1, pp. 1–48, 2015.

[37] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.

[38] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.

[39] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[40] R. Chalapathy, A. K. Menon, and S. Chawla, "Anomaly detection using one-class neural networks," arXiv:1802.06360, 2018.

[41] S. M. Erfani, S. Rajasegarar, S. Karunasekera, and C. Leckie, "High-dimensional and large-scale anomaly detection using a linear one-class SVM with deep learning," *Pattern Recognition*, vol. 58, pp. 121–134, 2016.

[42] L. Ruff, R. Vandermeulen, N. Görnitz, et al., "Deep one-class classification," in *Proc. 35th Int. Conf. Machine Learning*, 2018.